

**Министерство науки и высшего образования Российской
Федерации**

**ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«Национальный исследовательский университет ИТМО»**

Факультет программной инженерии и компьютерной техники

Курсовая работа

**Тема: Разработка Android-приложения с
использованием архитектуры MVVM, REST API,
локального хранилища Room и фоновой
синхронизации**

Выполнил:

Шайтор Илья

Проверил:

Шатинский Григорий Сергеевич

Санкт-Петербург

2026 г.

ВВЕДЕНИЕ	3
1. ОБЗОР ТЕХНОЛОГИЙ И ИНСТРУМЕНТОВ	4
1.1 Среда разработки	4
1.2 Архитектурные паттерн	4
1.3 Используемые библиотеки	5
2. РЕАЛИЗАЦИЯ ПРИЛОЖЕНИЯ	7
2.1 Навигация в приложении.	7
2.2 Функционал смены темы	8
2.3 Реализация функционала загрузки новостей	9
2.3.1 Реализация взаимодействия с API	9
2.3.2 Реализация взаимодействия с БД	10
2.3.3 Реализация репозитория	12
2.3.4 Реализация работы с данными на стороне интерфейса	14
2.3.5 Отображение данных	15
2.3.6 Реализация адаптера	17
2.4 Функционал лайков	18
2.5 Фоновый процесс	20
2.5.1 Запрос разрешений	21
2.5.2 Обновление сообщения	21
2.5.3 Отправка уведомлений	23
ЗАКЛЮЧЕНИЕ	25
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	26

ВВЕДЕНИЕ

Цель работы: разработать и продемонстрировать прототип мессенджера с архитектурой MVVM, устойчивым хранением состояния и поддержкой сетевого взаимодействия и фоновых задач. Приложение должно быть построено с чётким разделением ответственности, согласно принципам профессиональной разработки Android-приложений.

В рамках курсовой работы было реализовано Android приложение, содержащее в себе следующие элементы:

- Панель навигации
- Смена темы
- Запрос разрешений
- Работа с внешним API, загрузка постов
- Работа с локальной БД, сохранение постов
- Фоновый WorkManager, автообновление постов
- Отправка уведомлений о загрузке постов
- Возможность ставить “лайки” на посты

Приложение реализует архитектурный паттерн MVVM, разделяющий приложение на разные зоны ответственности и обеспечивающий открытость к модификации приложения.

1. ОБЗОР ТЕХНОЛОГИЙ И ИНСТРУМЕНТОВ

1.1 Среда разработки

В качестве основной среды разработки (IDE) была выбрана Android Studio, которая является официальным и наиболее востребованным инструментом для создания Android-приложений.

В качестве основного языка программирования был выбран Kotlin. Это обусловлено его современным синтаксисом и полной совместимостью с экосистемой Android.

Для управления зависимостями и автоматизации процесса сборки проекта использовалась система Gradle. Данный инструмент позволяет гибко настраивать процесс компиляции, подключать внешние библиотеки и управлять версиями зависимостей в одном месте. Использование Gradle в проекте обеспечит модульность структуры и упростит процесс развертывания готового приложения на различных устройствах.

1.2 Архитектурные паттерн

Выбор данной архитектуры обусловлен необходимостью четкого разделения ответственности между логикой обработки данных и интерфейсом пользователя, что соответствует принципам SOLID.

Основные компоненты архитектуры:

- **Model:** Отвечает за работу с данными. В данном слое описываются источники данных.
- **View:** В роли View выступают Fragments. Ответственность этого слоя ограничивается отображением данных и пересылкой действий пользователя в ViewModel. View «подписывается» на изменения состояния, что позволяет интерфейсу обновляться автоматически.
- **ViewModel:** Является связующим звеном. Она запрашивает данные у Model и подготавливает их для вывода во View. Главное преимущество ViewModel в Android — сохранение состояния при повороте экрана.

Для обеспечения эффективного управления данными в приложении был реализован паттерн Repository.

Использование слоя репозитория позволяет изолировать логику обработки данных и упрощает замену источников данных без изменения кода пользовательского интерфейса, поскольку ViewModel работает независимо от источника данных.

Также для обеспечения актуальности информации в приложении реализован модуль сетевого взаимодействия. API клиент инкапсулирует логику формирования HTTP-запросов, обработки заголовков и интерпретации ответов сервера.

И также в приложении используется простая реализация DI для снижения связанности компонентов. Это позволяет избежать создания экземпляров классов внутри других компонентов. Таким образом во ViewModel передаётся объект репозитория уже со своими зависимостями.

А для выполнения фоновых операций, был использован компонент WorkManager. Данный инструмент автоматически учитывает системные ограничения оптимизируя потребление ресурсов устройства.

1.3 Используемые библиотеки

Пользовательский интерфейс построен на основе Material Design 3 - актуальной итерацией дизайн-системы от компании Google.

Используемые компоненты:

- **MaterialCardView:** Использован для создания визуальной иерархии и структурирования контента в виде карточек с тенями и скруглениями для отображения загруженных постов.
- **FloatingActionButton (FAB):** Использован для сохранения положения кнопки загрузки постов
- **BottomNavigationView:** Реализован для обеспечения удобной и быстрой навигации между основными разделами приложения нажатием или свайпом.

Для реализации локального хранилища данных была использована библиотека Room, входящая в состав Android Jetpack, поскольку она является объектно-реляционной надстройкой над SQLite. Более тяжёлой базы для задач приложения не требовалось. Применение Room позволяет проверять SQL-запросы на этапе компиляции, что гарантирует стабильность работы с базой данных.

Для организации же взаимодействия с внешним API была выбрана библиотека Retrofit. Она представляет собой тип-безопасный HTTP-клиент, который абстрагирует низкоуровневые детали сетевых запросов. Использование Retrofit позволило декларативно описывать структуру запросов через интерфейсы, что значительно упростило поддержку кода и снизило вероятность ошибок при работе с сетевыми протоколами. Библиотека также обеспечивает автоматическую десериализацию JSON-ответов в объекты Kotlin и наоборот, что ускоряет обработку данных и упрощает разработку.

2. РЕАЛИЗАЦИЯ ПРИЛОЖЕНИЯ

2.1 Навигация в приложении.

Навигация в приложении реализована с использованием компонента Navigation из Android Jetpack. Приложение состоит из трёх основных экранов, доступных через нижнюю панель навигации:

1. Лента сообщений (NewsFragment) — главный экран с новостной лентой;
2. Профиль (ProfileFragment) — информация о пользователе;
3. Настройки (SettingsFragment) — параметры приложения (на данном этапе реализации только смена темы)

Код навигационного графа следующий:

```
<?xml version="1.0" encoding="utf-8"?>
<navigation
xmlns:android="http://schemas.android.com/apk/res/a
ndroid"

xmlns:app="http://schemas.android.com/apk/res-auto"
    android:id="@+id/nav_graph"
    app:startDestination="@id/ProfileFragment">

    <fragment
        android:id="@+id/ProfileFragment"

        android:name="com.example.mobiledevslb1.fragments.P
rofileFragment"
        android:label="Profile"/>

    <fragment
        android:id="@+id/NewsFragment"

        android:name="com.example.mobiledevslb1.fragments.N
ewsFragment"
        android:label="News"/>

    <fragment
```

```

        android:id="@+id/SettingsFragment"

        android:name="com.example.mobiledevslb1.fragments.SettingsFragment"
        android:label="Settings"/>
    </navigation>

```

Листинг 1: навигационный граф nav_graph.xml

Для реализации графа в MainActivity используется BottomNavigationView

```

val bottomNavigationView =
    findViewById<BottomNavigationView>(R.id.bottom_navigation)
val navController =
    findNavController(R.id.nav_host_fragment)
NavigationUI.setupWithNavController(bottomNavigationView, navController)

```

Листинг 2: навигация в MainActivity

2.2 Функционал смены темы

Смена темы реализована через отслеживания состояния переменной в SettingViewModel.

```

private val _isDarkTheme = MutableLiveData(false)
val isDarkTheme: LiveData<Boolean> = _isDarkTheme

fun changeIsDarkTheme() {
    _isDarkTheme.value = !(_isDarkTheme.value ?: false)
    Log.d("ViewModel", "Called SettingViewFragment.isDarkTheme setter")
}

```

Листинг 3: изменение состояния переменной

И изменения темы в SettingFragment при изменении переменной, путём подписки:

Использование LiveData позволяет реализовать подписку на изменения в SettingFragment, что обеспечивает мгновенное обновление UI-компонентов сразу после переключения триггера пользователем:

```
val changeThemeButton =
view.findViewById<Button>(R.id.changeThemes)
changeThemeButton.setOnClickListener {
    _viewModel.changeIsDarkTheme()
}

_viewModel.isDarkTheme.observe(viewLifecycleOwner)
{ isDark ->
    val currentTheme =
AppCompatActivity.getDefaultNightMode()
    if (
        (currentTheme ==
AppCompatActivity.MODE_NIGHT_YES && !isDark) ||
        (currentTheme ==
AppCompatActivity.MODE_NIGHT_NO && isDark) ||
        (currentTheme == -100)
    )
    {
        AppCompatActivity.setDefaultNightMode(
            if (isDark)
AppCompatActivity.MODE_NIGHT_YES
            else AppCompatActivity.MODE_NIGHT_NO
        )
        Log.d("ViewModel", "Theme changed")
    }
}
```

Листинг 4: изменение интерфейса при смене значения переменной

2.3 Реализация функционала загрузки новостей

2.3.1 Реализация взаимодействия с API

Для имитации внешнего API использовался сервис jsonplaceholder. Он позволяет получать “посты” и комментарии к ним. Приложение загружает именно комментарии.

Для взаимодействия с внешним API настраивался ApiClient на база Retrofit.

```
private val retrofit = Retrofit.Builder()
    .baseUrl("https://jsonplaceholder.typicode.com/")
    .client(httpClient)
    .addConverterFactory(GsonConverterFactory.create())
    .build()
```

Листинг 5: настройка retrofit

В качестве httpClient OkHttpClient с настроенным логгером. Элемент GsonConverterFactory необходим для сериализации и десериализации ответов API.

2.3.2 Реализация взаимодействия с БД

Полученные в виде dto данные преобразуются в Entity и хранятся в базе данных, реализованной с помощью библиотеки Room. Entity хранит в себе информацию, полученную из внешнего API, а также был ли пост отмечен пользователем.

```
@Entity(tableName = "messages")
data class MessageEntity(
    @PrimaryKey val id: Int,
    val author: String,
    val text: String,
    val isLiked: Boolean = false
)
```

Листинг 6: сущность для базы данных

Поля таблицы:

- id — уникальный идентификатор сообщения (первичный ключ);
- text — текст сообщения или URL;
- author — имя автора;
- isLiked — флаг «лайка» (сохраняется локально);

Отправка запросов в базу данным происходит с помощью настроенного объекта DAO. В нём настроены запросы для получения всех

сообщений (нужно для загрузки сообщений в фрагменты интерфейса), запрос для вставки данных (с заменой уже существующих), а также очистки всей базы. Запросы, связанные с функционалом лайков будут рассмотрены ниже.

```
@Dao
interface MessageDao {

    @Query("SELECT * FROM messages ORDER BY id
DESC")
    suspend fun getMessagesAll(): Flow<List<MessageEntity>>

    @Insert(onConflict = OnConflictStrategy.REPLACE)
    suspend fun insertAll(messages:
List<MessageEntity>)

    @Query("DELETE FROM messages")
    suspend fun clearAll()

    @Query("UPDATE messages SET isLiked = :isLiked
WHERE id = :id")
    suspend fun updateLikeStatus(id: Int, isLiked:
Boolean)

    @Query("SELECT id FROM messages WHERE isLiked =
1")
    suspend fun getLikedIds(): List<Int>
}
```

Листинг 7: настройка DAO

Также стоит отметить следующее:

- `getMessagesAll()` возвращает `Flow` для реактивного обновления UI;
- все операции помечены как `suspend` для использования с корутинами.

Сама база настроена следующим образом:

```

@Database(
    entities = [MessageEntity::class],
    version = 2,
    exportSchema = false
)
abstract class AppDatabase : RoomDatabase() {

    abstract fun messageDao(): MessageDao

    companion object {
        @Volatile private var INSTANCE: AppDatabase?
        = null

        fun getInstance(context: Context):
AppDatabase =
            INSTANCE ?: synchronized(this) {
                INSTANCE ?: Room.databaseBuilder(
                    context.applicationContext,
                    AppDatabase::class.java,
                    "messages.db"
                )

                .fallbackToDestructiveMigration()
                    .build().also { INSTANCE = it }
            }
    }
}

```

Листинг 8: настройка инициализации и подключения к базе данных

Для избежания повторной инициализации объекта взаимодействия с базой, перед его инициализацией происходит проверка существования данного объекта.

2.3.3 Реализация репозитория

Для объединения элементов и обеспечения их общения с интерфейсом реализован репозиторий. Он хранит в себе сообщения как Flow объект, преобразуя данные в модель для использования этих данных на стороне интерфейса, а также для их реактивного обновления.

Также в репозитории содержится функция для обновления сообщений, работающая по следующему алгоритму:

- сообщения загружаются с внешнего API
- они преобразуются в сущности (с сохранением состояния лайков, об этом подробнее ниже)
- данные обновляются в базе данных
- поскольку данные обновились, то все подписчики на переменную message получили уведомления об этом

```
val messages: Flow<List<Message>> =
    dao.getMessageAll().map { list ->
        list.map { it.toDomain() }
    }

suspend fun refreshMessages(): Result<Unit> {
    return try {
        Log.d("REPO", "Начинаю качать...")
        val dto = api.getMessage()
        Log.d("REPO", "Скачал ${dto.size} объектов.
Начинаю маппинг...")
        val likedIds = dao.getLikedIds().toSet()
        Log.d("REPO", "Найдено лайков в БД:
${likedIds.size}")
        val entities = dto.map { item ->
            val entity = item.toEntity()
            if (likedIds.contains(entity.id)) {
                entity.copy(isLiked = true)
            } else {
                entity
            }
        }
        Log.d("REPO", "Маппинг готов. Записываю в
БД...")
        dao.insertAll(entities)
        Log.i("MessageRepository", "Loaded
${entities.size} messages from API")

        Result.success(Unit)
    } catch (e: Exception) {
        Log.e("MessageRepository", "Failed to
```

```
refresh", e)
    Result.failure(e)
}
}
```

Листинг 9: функционал репозитория

2.3.4 Реализация работы с данными на стороне интерфейса

В ViewModel инициализируется репозиторий с помощью простой реализации DI, а также хранится подписка на данные репозитория. Помимо этого прокидывается функция для кнопки обновления данных (loadMessages).

```
private val repository: MessageRepository =
    GetService.messageRepository(application)
val messages: Flow<List<Message>> =
    repository.messages

fun loadMessages() {
    viewModelScope.launch {
        repository.refreshMessages()
    }
}
```

Листинг 10: управление данными во ViewModel

DI реализовано следующим образом:

```
object GetService {
    fun messageRepository(context: Context):
    MessageRepository {
        val database =
    AppDatabase.getInstance(context)
        val dao = database.messageDao()
        val api = ApiClient.messageApi

        return MessageRepository(api, dao)
    }
}
```

Листинг 11: использование DI

2.3.5 Отображение данных

На стороне View подключён адаптер, поскольку каждый пост распакован с помощью RecyclerView. Для реактивного обновления интерфейса на данные ViewModel оформлена подписка. А на кнопку обновления данных привязана функция на клик.

```
private val viewModel: NewsViewModel by
viewModelProviders()

override fun onCreateView(view: View,
savedInstanceState: Bundle?) {
    super.onCreateView(view, savedInstanceState)

    val recyclerView: RecyclerView =
view.findViewById(R.id.rvMessages)
    val btnDownload: FloatingActionButton =
view.findViewById(R.id.btnDownload)

    val adapter = MessageAdapter(onLikeClick = {
message ->
    viewModel.onLikeClicked(message)
    })
    recyclerView.adapter = adapter
    recyclerView.layoutManager =
LinearLayoutManager(requireContext())

    btnDownload.setOnClickListener {
        viewModel.loadMessages()
    }

    viewLifecycleOwner.lifecycleScope.launch {
        viewModel.messages.collect { list ->
            adapter.submitList(list)
        }
    }
}
```

Листинг 12: обновление данных в View

На стороне .xml для распаковки постов используется RecyclerView для распаковки элементов и FloatingActionButton для отображения кнопки загрузки постов.

```
<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/rvMessages"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:clipToPadding="false"
    android:paddingBottom="80dp"

    app:layout_behavior="@string/appbar_scrolling_view_
behavior" />

<com.google.android.material.floatingactionbutton.F
loatingActionButton
    android:id="@+id/btnDownload"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_gravity="bottom|end"
    android:layout_margin="16dp"
    android:layout_marginBottom="70dp"

    app:backgroundTint="@color/design_default_color_pri
mary"

    app:srcCompat="@android:drawable/stat_sys_download"
/>
```

Листинг 13: экран с новостями

Сама реализация карточки содержит в себе элементы TextView для отображения автора и текста, ImageButton для отображения иконки лайка и ShapeableImageView для аватарки.

```

<com.google.android.material.imageview.ShapeableImage
view
    android:id="@+id/ivAvatar"
    android:layout_width="50dp"
    android:layout_height="50dp"
    android:scaleType="centerCrop"
    android:background="#DDDDDD"
    app:shapeAppearanceOverlay="@style/CircleImage"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toTopOf="parent"
    tools:src="@tools:sample/avatars" />

```

Листинг 14: объект для хранения аватарки сообщения

2.3.6 Реализация адаптера

Для обеспечения высокой производительности интерфейса при работе с динамическими массивами данных в адаптере реализованы специализированные классы `MessageViewHolder` и `MessageDiffCallback`.

Использование паттерна `ViewHolder` позволяет кэшировать ссылки на графические компоненты, исключая избыточные вызовы метода `findViewById()`. Класс `MessageDiffCallback` реализует алгоритмы эффективного сравнения элементов, благодаря чему обновление интерфейса происходит точно — только для тех данных, которые подверглись изменениям. Механизм рециклинга (переиспользования) в `RecyclerView` гарантирует, что в памяти системы создается и хранится лишь необходимый минимум объектов `View` (текущие видимые элементы и небольшой резерв), что значительно оптимизирует потребление ресурсов устройства.

```

class MessageViewHolder(itemView: View) :
    RecyclerView.ViewHolder(itemView) {
    private val tvAuthor: TextView =
        itemView.findViewById(R.id.tvAuthor)
    private val tvBody: TextView =
        itemView.findViewById(R.id.tvBody)
    private val ivAvatar: ImageView =

```

```

itemView.findViewById(R.id.ivAvatar)
    private val btnLike: ImageButton =
itemView.findViewById(R.id.btnLike)

    fun bind(
        message: Message,
        onLikeClick: (Message) -> Unit
    ) {
        tvAuthor.text = message.author
        tvBody.text = message.text

ivAvatar.setImageResource(R.drawable.ic_launcher_fo
reground)

        val iconRes = if (message.isLiked)
R.drawable.ic_liked else R.drawable.ic_unliked
        btnLike.setImageResource(iconRes)

        btnLike.setOnClickListener {
            onLikeClick(message)
        }
    }
}

class MessageDiffCallback :
DiffUtil.ItemCallback<Message>() {
    override fun areItemsTheSame(oldItem: Message,
newItem: Message): Boolean {
        return oldItem.id == newItem.id
    }

    override fun areContentsTheSame(oldItem:
Message, newItem: Message): Boolean {
        return oldItem == newItem
    }
}

```

Листинг 15: код адаптера сообщений

2.4 Функционал лайков

Функционал лайков со стороны пользователя реализован как изменения иконки лайка при нажатии на неё. На стороне View в адаптер прокинута функция клика на кнопку.

```
val adapter = MessageAdapter(onLikeClick = {
    message ->
        viewModel.onLikeClicked(message)
})
```

Листинг 16: передача в адаптер функции onLikeClicked

В сам адаптер ставит эту функцию на прослушку события клика, а также привязывает смену ресурса иконки на изменения поля в базе данных

```
val iconRes = if (message.isLiked)
    R.drawable.ic_liked else R.drawable.ic_unliked
btnLike.setImageResource(iconRes)

btnLike.setOnClickListener {
    onLikeClick(message)
}
```

Листинг 17: привязка функции к кнопке внутри карточки сообщения

Сама функция клика описана в ViewModel. Функция вызывает метод репозитория, который уже обновляет данные в базе.

```
val messages: Flow<List<Message>> =
    repository.messages

fun onLikeClicked(message: Message) {
    viewModelScope.launch {
        Log.d("toggleLike", "Клик прочитан
        ${message}")
        repository.toggleLike(message)
    }
}
```

Листинг 18: реализация функции нажатия на клик со стороны интерфейса

В самом репозитории функционал лайка просто меняет значение булевого поля базы данных.

```
suspend fun toggleLike(message: Message) {
    dao.updateLikeStatus(message.id,
!message.isLiked)
}
```

Листинг 19: реализация функции нажатия на клик со стороны репозитория

Интересной деталью реализации является сохранение состояния при перезагрузке данных с внешнего API. Это происходит благодаря тому, что перед перезаписью данных из базы берутся старые записи и состояние поля `isLiked` передаётся при маппинге `dto` представление в `entity` представление.

```
val likedIds = dao.getLikedIds().toSet()
Log.d("РЕПО", "Найдено лайков в БД:
${likedIds.size}")

val entities = dto.map { item ->
    val entity = item.toEntity()
    if (likedIds.contains(entity.id)) {
        entity.copy(isLiked = true)
    } else {
        entity
    }
}
```

Листинг 20: реализация сохранения состояния лайка сообщения при перезагрузке данных с внешнего API

2.5 Фоновый процесс

Для автоматического обновления сообщений настроен фоновый процесс, а для уведомления пользователя при успешной отработке процесса присылается уведомление.

2.5.1 Запрос разрешений

При запуске приложения происходит проверка наличия разрешений и, в случае их отсутствия, пользователю отображается запрос на предоставления разрешений.

```
override fun onCreate(savedInstanceState: Bundle?)
{
    super.onCreate(savedInstanceState)
    setContentView(R.layout.main_activity_layout)

    SyncWorker.schedule(this)

    requestAppPermissions()
}
```

Листинг 21: запуск фоновой службы и проверка разрешений при создании MainActivity

```
private fun requestAppPermissions() {
    if (Build.VERSION.SDK_INT >=
        Build.VERSION_CODES.TIRAMISU) {
        if (ContextCompat.checkSelfPermission(
            this,
            Manifest.permission.POST_NOTIFICATIONS,
            ) != PackageManager.PERMISSION_GRANTED
        ) {
            ActivityCompat.requestPermissions(
                this,
                arrayOf(Manifest.permission.POST_NOTIFICATIONS),
                REQ_NOTIF,
            )
        }
    }
}
```

Листинг 22: запрос прав на отправку уведомлений

2.5.2 Обновление сообщения

Права на отправку уведомлений нужны, чтобы уведомлять пользователя об обновлении сообщений. На листинге 21 присутствует строка запуска фоновой службы. Сам процесс реализован в отдельном

файле в классе SyncWorker: CoroutineWorker. Метод doWork() запускает метод репозитория и отправку уведомлений.

```
override suspend fun doWork(): Result {
    Log.d("SyncWorker", "doWork запущен")

    return try {
        val repository =
            GetService.messageRepository(applicationContext as
            Application)

        repository.refreshMessages()

        showNotification("Обновление постов",
            "Успешно")

        Log.d("SyncWorker", "doWork отработал")
        Result.success()
    } catch (e: Exception) {
        Log.e("SyncWorker", "doWork упал", e)
        Result.retry()
    }
}
```

Листинг 23: функционал фоновой службы

Объект процесса создаётся как companion object. Время периодичности отработки устанавливается минимальное, а именно 15 минут.

```
companion object {
    fun schedule(context: Context) {
        val constraints = Constraints.Builder()

        .setRequiredNetworkType(NetworkType.CONNECTED)
        .build()

        val request =
            PeriodicWorkRequestBuilder<SyncWorker>(
                15, TimeUnit.MINUTES,
            )
            .setConstraints(constraints)
            .build()
    }
}
```

```

WorkManager.getInstance(context).enqueueUniquePeriodicWork(
    "SyncMessagesWork",
    androidx.work.ExistingPeriodicWorkPolicy.KEEP,
    request,
)
}
}

```

Листинг 24: объект службы

2.5.3 Отправка уведомлений

Одним из ещё не рассмотренных функционалов осталась отправка уведомлений пользователю при успешном выполнении процесса. Отправка уведомлений реализована с помощью встроенной реализации. В менеджере создаётся канал, затем с помощью билдера NotificationCompat создаётся сам объект уведомления, который и отправляется пользователю.

```

private fun showNotification(title: String, text:
String) {
    val channelId = "news_channel"

    val manager =
applicationContext.getSystemService(Context.NOTIFIC
ATION_SERVICE)
        as NotificationManager

    if (Build.VERSION.SDK_INT >=
Build.VERSION_CODES.O) {
        val channel = NotificationChannel(
            channelId,
            "Синхронизация сообщений",
            NotificationManager.IMPORTANCE_DEFAULT,
        )
        manager.createNotificationChannel(channel)
    }
}

```

```
    val notification =  
NotificationCompat.Builder(applicationContext,  
channelId)  
    .setSmallIcon(R.drawable.ic_notification)  
    .setContentTitle(title)  
    .setContentText(text)  
    .setAutoCancel(true)  
    .build()  
  
manager.notify(1001, notification)  
}
```

Листинг 25: функция отправки уведомлений

ЗАКЛЮЧЕНИЕ

В результате выполнения курсовой работы было разработано Android-приложение способное получать, отображать и хранить различные сообщения с внешнего API, а также автоматически обновлять информацию и уведомлять об этом пользователя.

Приложение использует современные подходы к мобильной разработке, используя пользовательский интерфейс на Material Design 3, архитектуру MVVM.

Построенная архитектура обеспечивает хорошую масштабируемость и удобство поддержки: логику работы с данными можно расширять без существенной переработки UI. Реализованные механизмы хранения данных позволяют использовать приложение даже без доступа к интернету.

В качестве возможного расширения функционала можно отметить:

- Добавление авторизации
- Реализация отправки, а не только получения сообщений;
- Поиск, фильтрация и пагинация для больших списков;
- Покрытие проекта unit- и UI-тестами, настройка CI/CD;

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Android Developers. Documentation. - URL: <https://developer.android.com/> (дата обращения: 25.01.2026).
2. Android Developers. Room Persistence Library. - URL: <https://developer.android.com/training/data-storage/room> (дата обращения: 25.01.2026).
3. Android Developers. WorkManager. - URL: <https://developer.android.com/topic/libraries/architecture/workmanager> (дата обращения: 25.01.2026).
4. Android Developers. Navigation component. - URL: <https://developer.android.com/guide/navigation> (дата обращения: 25.01.2026).
5. Kotlin Documentation. - URL: <https://kotlinlang.org/docs/home.html> (дата обращения: 25.01.2026).
6. Square. Retrofit. - URL: <https://square.github.io/retrofit/> (дата обращения: 25.01.2026).
7. Square. OkHttp. - URL: <https://square.github.io/okhttp/> (дата обращения: 25.01.2026).
8. Material Design 3. - URL: <https://m3.material.io/> (дата обращения: 25.01.2026).
9. JSONPlaceholder. Free fake API for testing and prototyping. - URL: <https://jsonplaceholder.typicode.com/> (дата обращения: 23.01.2026).